



MudRYder - Personnage

PROJET FINAL - MUDRYDER

Gabriel Zeizer, Aurélien Santi

11 juin 2026

Table des matières

1 – Introduction	3
2 – Moyens à disposition	3
2.1 – GitHub	3
2.2 – GDX2D	3
3 – Inspiration	4
4 – Choix d'implémentation	4
4.1 – Architecture générale	4
4.2 – Gestion des modes de jeu	5
4.3 – Deux systèmes de dessin distincts	5
4.4 – Hiérarchie des lignes	6
4.5 – Gestion de la caméra	6
4.6 – Optimisation du rendu	6
4.7 – Système de sauvegarde et chargement	7
4.8 – Détection de collision pour l'effacement	7
4.9 – Gestion audio	7
5 – Problèmes rencontrés et solutions	8
5.1 – Surcharge CPU	8
5.2 – Crash avec des tableaux de lignes vides	8
5.3 – Gestion des angles du personnage	8
5.4 – Optimisation des calculs vectoriels	8
6 – Fonctionnalités	8
6.1 – Ce qui fonctionne	8
6.2 – Limites connues	8
7 – Améliorations et auto-critique	9
7.1 – Propositions d'améliorations	9
7.2 – Auto-critique	9
8 – Conclusion	9
9 – Annexes	9
9.1 – Point d'entrée	9
9.2 – Cœur du jeu	9
9.3 – Systèmes de dessin	17
9.4 – Types de lignes	22
9.5 – Fenêtres UI	23
9.6 – Modes de jeu	27
9.7 – Types	29
Références	29

1 – Introduction

Ce projet a été effectué dans le cadre du cours de Programmation Orientée-Objets. Il s'agissait d'un projet plutôt libre dont l'objectif était de créer un jeu vidéo afin de mettre en oeuvre les concepts acquis tout au long de l'année.

Nous avons choisi de recréer le jeu **Line Rider**, un classique du genre « bac à sable » où le joueur dessine des pistes qu'un personnage parcourt ensuite par gravité.

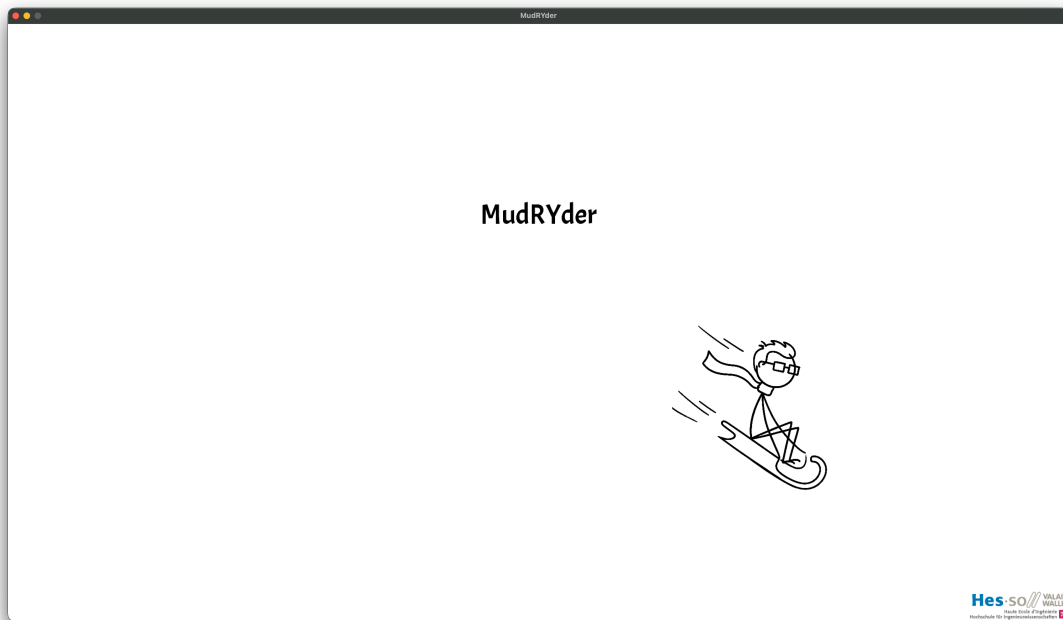


Figure 1 - Écran de démarrage du jeu

2 – Moyens à disposition

Afin d'assurer le bon déroulement de ce projet, nous avons utilisé la bibliothèque GDX2, une bibliothèque Java simple, sur laquelle nous nous sommes appuyé afin de gérer la partie graphique et physique. Le projet a été entièrement écrit en Scala 2.13.18 à l'aide d'IntelliJ IDEA.



Figure 3 - Logo de Scala

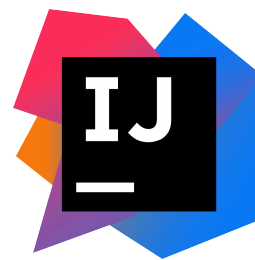


Figure 4 - Logo de IntelliJ IDEA

2.1 – GitHub

Ce projet étant commun, l'utilisation de Git nous a permis de collaborer de manière efficace et rapide. Nous avons utilisé GitHub pour sa simplicité d'intégration dans notre processus de développement.

Le projet est disponible à l'adresse suivante : <https://github.com/swiss-vsdev/MudRYder>

2.2 – GDX2D

La bibliothèque GDX2D est un projet open-source créée par divers enseignants de l'HES-SO Valais/Wallis. Le projet a été basé sur une bibliothèque déjà existante : libGDX [1].

3 – Inspiration

Nous avons eu l'idée de nous lancer dans la création de ce jeu en nous inspirant de Line Rider [2], un jeu de style bac à sable dans lequel un rider glisse le long d'une ligne dessinée par le joueur. Ce type de jeu est très différent de la majorité : il n'a pas de gagnant ou de perdant, mais il en va de la créativité de chacun.

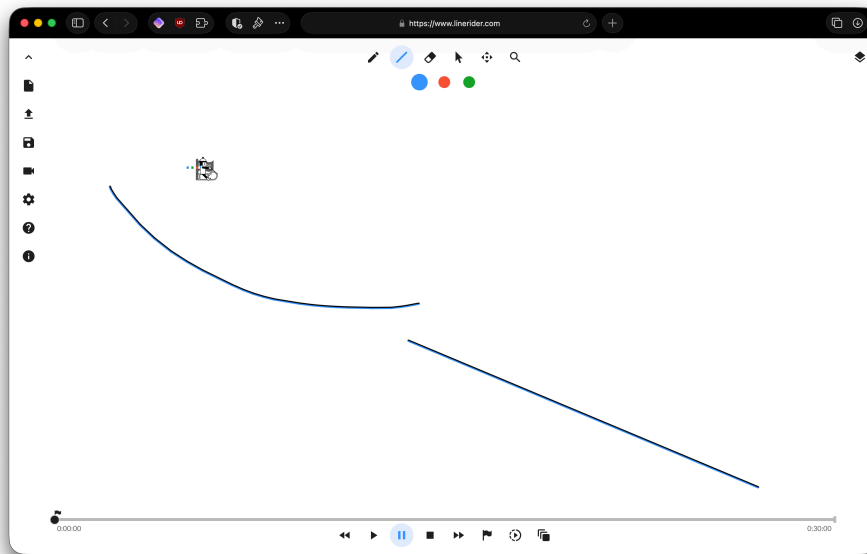


Figure 5 - Line Rider, le jeu original

4 – Choix d'implémentation

4.1 – Architecture générale

Le projet est structuré autour d'une classe principale `Game` qui étend `DesktopApplication` de la bibliothèque GDX2D. Cette classe gère la boucle de jeu principale et délègue les différentes fonctionnalités à des sous-systèmes spécialisés : `LineDrawMachine` et `FreeDrawMachine` pour le dessin, `MenuModesMachine` pour la gestion des modes, `MudryMachine` pour le personnage, et `MusicPlayer` pour l'audio.

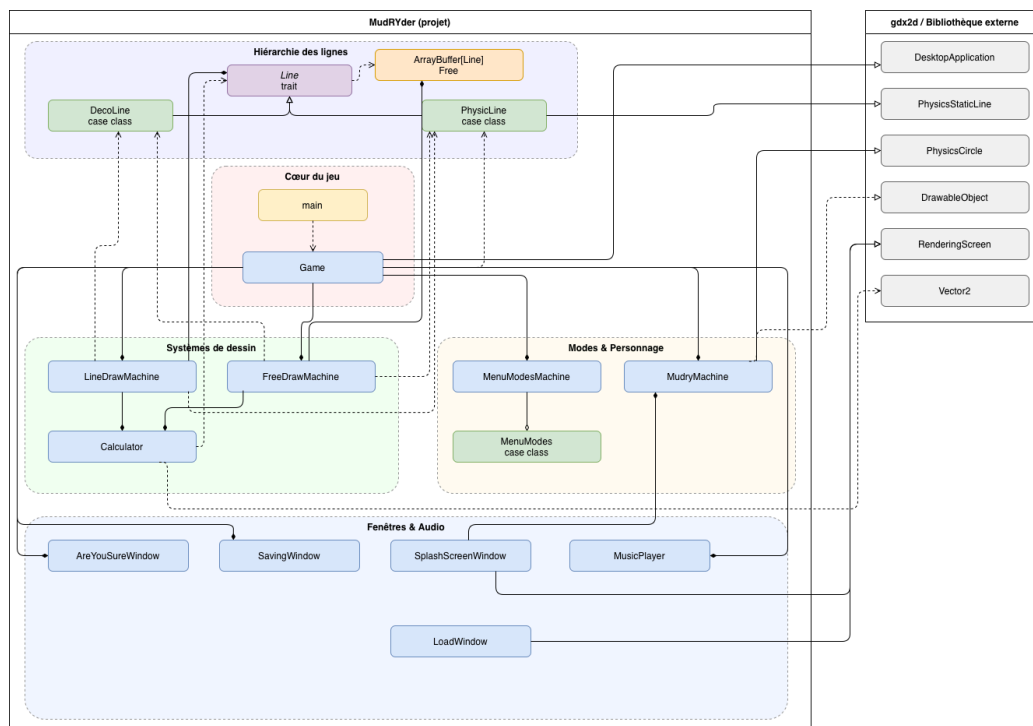


Figure 6 - Diagramme de classes UML du projet

4.2 – Gestion des modes de jeu

Nous avons implémenté une machine à états simple via la classe `MenuModesMachine`. Celle-ci gère les modes (`lines`, `free`, `play`, `eraser`, `mop`, `save`, `load`) ainsi que des toggles internes pour le mode de dessin (`physic` / `decoration`) et la musique (`music` / `musicmute`). Le mode courant est stocké sous forme de chaîne de caractères et transmis aux différentes machines de dessin, qui adaptent leur comportement en conséquence. Par exemple, en mode `play`, le dessin est désactivé et la caméra suit automatiquement le personnage.



Figure 7 - Menu des modes de jeu

4.3 – Deux systèmes de dessin distincts

Nous avons fait le choix de séparer le dessin de lignes segmentées (`LineDrawMachine`) et le dessin de lignes libres (`FreeDrawMachine`). La première fonctionne par clic, glisser et relâcher pour créer un segment rectiligne entre deux points, tandis que la seconde génère des traits continus en ajoutant des segments à chaque frame de glissement. Cette séparation s'explique par la différence fondamentale d'interaction utilisateur et de structure de données : `LineArray` pour les segments fixes, `FreeArray` (tableau de tableaux) pour les traits libres.

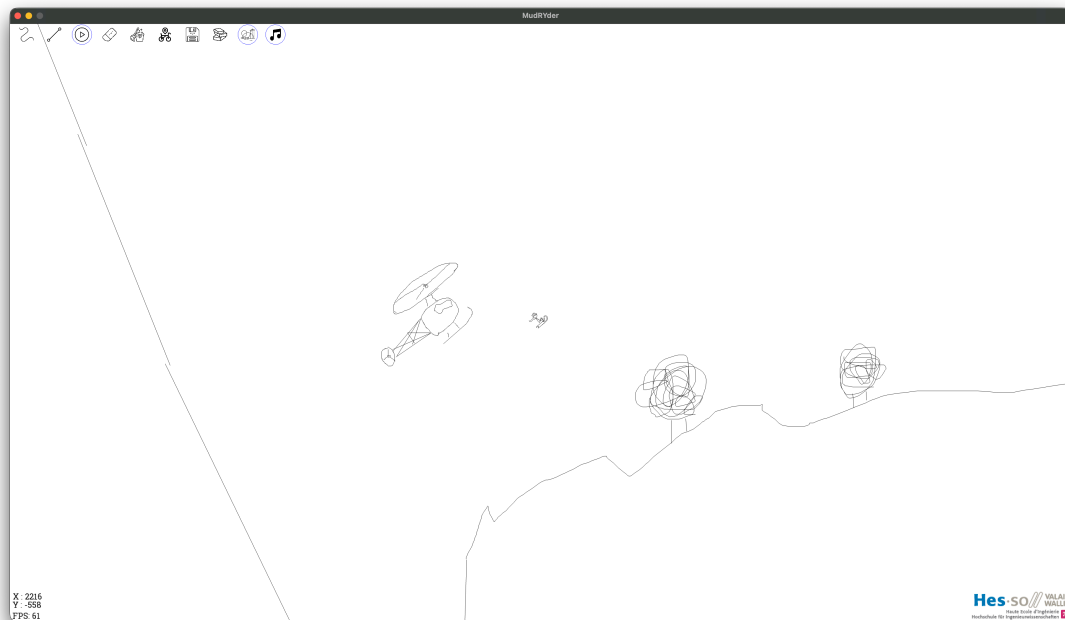


Figure 8 - Mudry en pleine course sur les pistes

4.4 – Hiérarchie des lignes

Nous avons défini un trait `Line` comme interface commune à toutes les lignes, imposant les propriétés `p1x`, `p1y`, `p2x`, `p2y` et les méthodes `draw` et `destroy`. Deux implémentations concrètes en `case class` en découlent :

- `PhysicLine` étend `PhysicsStaticLine` (corps physique `Box2D` [3]) et mixe le trait `Line`
- `DecoLine` implémente uniquement le trait `Line` sans composante physique

L'utilisation de `case class` nous permet de bénéficier de l'immuabilité des coordonnées, du pattern matching, et de l'égalité structurelle. L'héritage multiple de `PhysicLine` (classe physique + trait) a nécessité l'utilisation de `super[PhysicsStaticLine].destroy()` pour lever l'ambiguïté.

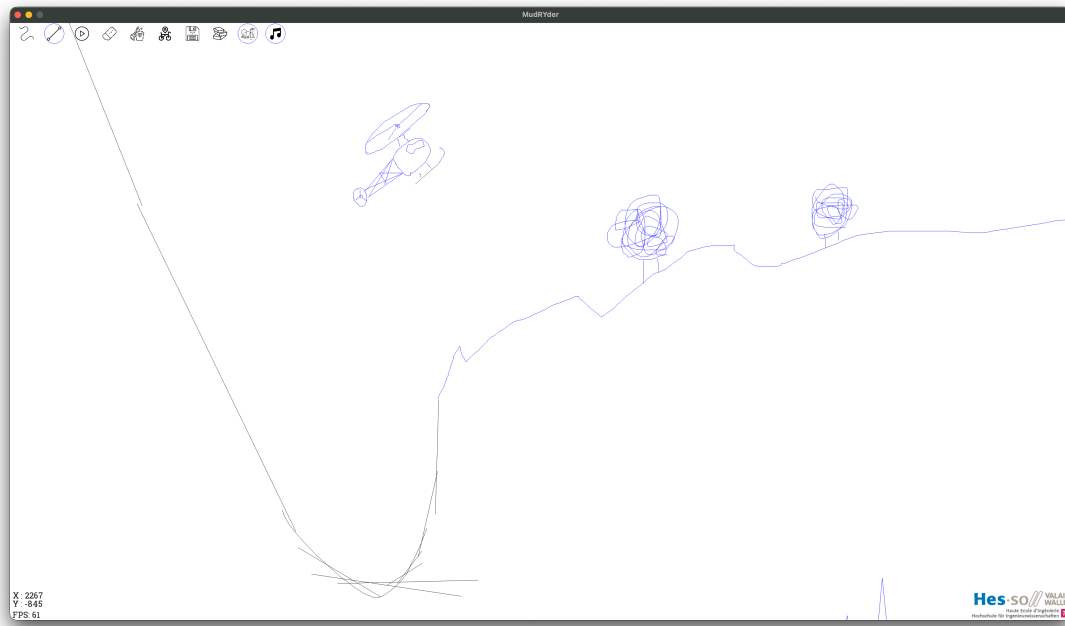


Figure 9 - Lignes physiques (noir) et décoratives (bleu)

4.5 – Gestion de la caméra

La caméra fonctionne selon deux modes. En mode édition (dessin), l'utilisateur peut se déplacer librement dans le monde via un glisser-déposer au clic droit, ce qui décale les coordonnées de la caméra (`camX`, `camY`). En mode `play`, la caméra suit automatiquement la position du personnage (`playerMachine.posX`, `playerMachine.posY`). Les coordonnées de clic souris sont converties en coordonnées monde via la formule $x + (camX - 960)$ pour l'axe X, ce qui permet d'interagir correctement avec les éléments même après un déplacement de la vue.

4.6 – Optimisation du rendu

Afin de maintenir des performances élevées, seules les lignes situées dans un rayon de 2000 pixels autour de la caméra sont dessinées. Cela nous a permis de réduire l'impact sur la performance plus le nombre de lignes augmente.

4.7 – Système de sauvegarde et chargement

Le format de sauvegarde choisi est le CSV, simple à lire et à écrire. Chaque ligne est enregistrée au format `TypeLigne,x1,y1,x2,y2` dans le dossier `./saves/`. Le nom de fichier est généré automatiquement à partir de la date et de l'heure (`save_JJJHHMMSS.csv`). Au chargement, le fichier est parsé et chaque ligne est reconstruite en `PhysicLine` ou `DecoLine` selon son type. Ce format texte présente l'avantage d'être lisible par un humain et facile à déboguer.

Voici un exemple de fichier de sauvegarde :

```
1 PhysicLine,100.0,200.0,500.0,200.0
2 DecoLine,300.0,400.0,600.0,400.0
3 PhysicLine,500.0,200.0,800.0,600.0
```

Listing 1 - Exemple de fichier de sauvegarde au format CSV

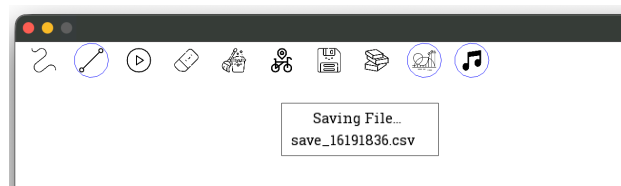


Figure 10 - Notification de sauvegarde

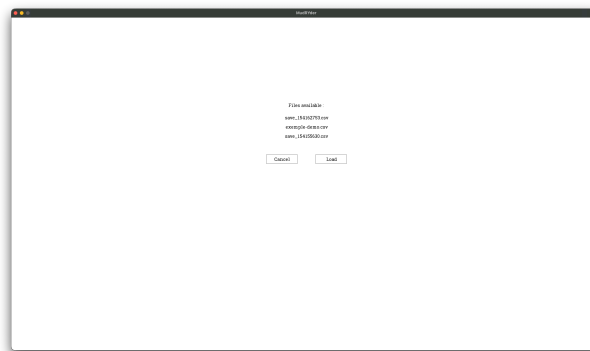


Figure 11 - Fenêtre de chargement

Chaque ligne contient : `TypeLigne`, `x1`, `y1`, `x2`, `y2`. Le type `PhysicLine` crée une ligne avec collision physique, `DecoLine` une ligne purement décorative (bleue, sans physique).

4.8 – Détection de collision pour l'effacement

La classe `Calculator` implémente un algorithme de distance point-segment basé sur le produit vectoriel. Cette méthode permet de déterminer si un clic est suffisamment proche d'une ligne (tolérance de 8 pixels) pour l'effacer. L'outil « gomme » parcourt toutes les lignes et supprime celles qui se trouvent dans le rayon du curseur, tandis que l'outil « serpillière » (`mop`) efface l'intégralité du dessin après confirmation de l'utilisateur via la fenêtre `AreYouSureWindow`.

4.9 – Gestion audio

Le `MusicPlayer` utilise l'API `javax.sound.sampled.AudioSystem` pour lire des fichiers WAV. Deux pistes distinctes sont utilisées : une pour le mode édition (`edit.wav`, en boucle) et une pour le mode jeu (`play.wav`, lecture unique). Le son peut être coupé via un bouton dans le menu, ce qui stoppe et ferme le clip audio pour libérer les ressources.

5 – Problèmes rencontrés et solutions

5.1 – Surcharge CPU

Au fur et à mesure que le nombre de lignes augmentait, le jeu commençait à ralentir considérablement. Le problème venait du fait que toutes les lignes étaient dessinées à chaque frame, même celles situées loin de la caméra. Nous avons résolu ce problème en implémentant un système où seules les lignes situées dans un rayon de 2000 pixels autour de la caméra sont dessinées. Cela a considérablement amélioré les performances.

5.2 – Crash avec des tableaux de lignes vides

Un crash survenait lorsque le joueur effaçait toutes les lignes puis tentait d'en dessiner une nouvelle. Le moteur physique ne gérait pas correctement un monde vide. Nous avons contourné le problème en insérant une ligne factice très loin de l'écran (coordonnées -10000, -10000) dès que le tableau est vide (*ArrayEmptyFix*).

5.3 – Gestion des angles du personnage

Initialement, le personnage ne suivait pas correctement l'inclinaison des pentes. Nous avons modifié la méthode `collision()` de *MudryMachine* pour lire l'angle du segment contacté et appliquer une correction au-delà de ± 120 degrés, stabilisant ainsi le comportement.

5.4 – Optimisation des calculs vectoriels

L'outil gomme utilisait des calculs de distance point-segment coûteux. Nous avons optimisé l'algorithme en ajoutant une vérification préalable des limites avant le calcul du produit vectoriel. Nous avons également essayé de réduire le nombre de cycles CPU nécessaires en modifiant certains calculs. En remplaçant un `math.pow(x,2)` par simplement `x * x` de cette manière, nous avons encore pu réduire l'impact des calculs.

6 – Fonctionnalités

6.1 – Ce qui fonctionne

- Dessin de lignes (segmentées et libres, physiques et décoratives)
- Physique complète (gravité, collisions avec le personnage)
- Caméra avec deux modes (suivi automatique en jeu, libre en édition)
- Outils d'effacement (gomme individuelle et serpillière avec confirmation)
- Sauvegarde et chargement de pistes au format CSV
- Écran de démarrage animé, musique d'ambiance, HUD (FPS, coordonnées)

6.2 – Limites connues

- La musique peut recommencer lors de certains événements, par exemple lors de l'exploration des fichiers enregistrés.
- La fenêtre de chargement suppose que le dossier `saves/` contient au moins un fichier valide
- Si un fichier est généré / ajouté au dossier `saves/` après avoir lancé le jeu, il n'apparaîtra pas.
- Le personnage ne suit pas parfaitement l'angle de la pente

7 – Améliorations et auto-critique

7.1 – Propositions d'améliorations

- Ajouter un outil « accélérateur » pour propulser Mudry (prévu dans la roadmap initiale)
- Rendre le déplacement sur les angles plus fluide
- Unifier les deux systèmes de dessin (`LineDrawMachine` et `FreeDrawMachine`) pour réduire la duplication de code
- Ajouter une gestion d'erreur (fichiers, index, dossier vide)
- Remplacer le correctif `ArrayEmptyFix` par une solution plus propre
- Améliorer la structure du code pour le rendre plus simple à consulter et modifier

7.2 – Auto-critique

Le projet s'est bien déroulé dans l'ensemble. L'utilisation de Git nous a permis de travailler en parallèle efficacement. Cependant, le découpage en classes aurait pu être mieux pensé en amont : la séparation en deux systèmes de dessin distincts a créé de la duplication et des incohérences, notamment pour la gomme et la sauvegarde. Nous avons passé beaucoup de temps à implémenter des fonctionnalités mais pas assez à restructurer le code et à l'optimiser. À l'avenir, il faudrait prioriser la robustesse du code avant les fonctionnalités secondaires.

8 – Conclusion

Ce projet nous a permis de mettre en pratique les concepts de programmation orientée-objet vus durant le semestre : héritage, polymorphisme, traits et les case classes. Malgré quelques limitations, le jeu est fonctionnel et atteint les objectifs fixés. Nous sommes satisfaits du résultat et cette expérience nous a appris l'importance d'une bonne architecture logicielle ainsi que d'une répartition claire des tâches au sein d'une équipe.

9 – Annexes

Le code source complet du projet est présenté ci-dessous.

9.1 – Point d'entrée

```

1 // Game entry point
2 // Have fun !
3 object main {
4   def main(args: Array[String]): Unit = {
5     new Game().launch()
6   }
7 }
```

Listing 2 - Point d'entrée du programme - main.scala

9.2 – Cœur du jeu

```

1 import ch.hevs.gdx2d.desktop.DesktopApplication
2 import ch.hevs.gdx2d.lib.physics.PhysicsWorld
3 import ch.hevs.gdx2d.lib.{GdxGraphics, ScreenManager}
4 import com.badlogic.gdx.Input
5 import com.badlogic.gdx.graphics.{Color, OrthographicCamera}
6 import com.badlogic.gdx.math.Vector2
7
8 import java.util.Calendar
9
10 // Main game class: handles rendering, input, camera, and mode switching
11 class Game extends DesktopApplication(1920, 1080) {
12   private val lineMachine = new LineDrawMachine
13   private val modesMachine = new MenuModesMachine
14   private val freeMachine = new FreeDrawMachine
```

```

15 private val uSure = new AreYouSureWindow
16 private val saveWin = new SavingWindow
17 private val walkman = new MusicPlayer
18 val s = new ScreenManager
19 private val s1 = new ScreenManager
20 private val startTime = System.currentTimeMillis()
21 private val l = PhysicLine(0, 100, 1920, 100)
22 private val l2 = PhysicLine(0, 1000, 1920, 100)
23 private var playerMachine: MudryMachine = _
24 private var onMenuClick: Boolean = false
25 private var iAmClicked: Boolean = false
26 var currentMode = ""
27 var musicMode = "music"
28 private var lastMouseClick = 1
29 private var cam = new OrthographicCamera
30 var camX: Int = 1920 / 2
31 var camY: Int = 540
32 private var RDragX: Int = 0
33 private var RDragY: Int = 0
34 private var lastMode = ""
35 private var currentX: Int = 0
36 private var currentY: Int = 0
37 private var wasMopping: Boolean = false
38 private var firstRun = true
39 private var isSaving = false
40 private var savefile = ""
41 private var saveTime: Long = _
42 private var drawZoneX: Int = 0
43 private var drawZoneY: Int = 0
44
45 // Initialises the player, window title, and overlay screens (splash, load)
46 override def onInit(): Unit = {
47     playerMachine = new MudryMachine("rider", new Vector2(960, 900), 4f, 0f, 0f, 0.00001f)
48     camX = playerMachine.posX.toInt
49     camY = playerMachine.posY.toInt
50     setTitle("MudRYder")
51     modesMachine.loadIcons()
52     s.registerScreen(classOf[SplashScreenWindow])
53     s1.registerScreen(classOf[LoadWindow])
54 }
55
56 // Main render loop: splash screen first, then camera, physics, drawing lines, and HUD
57 override def onGraphicRender(g: GdxGraphics): Unit = {
58     walkman.play(currentMode, musicMode)
59     // Show splash screen for the first 3 seconds
60     if (System.currentTimeMillis() - startTime < 3000) {
61         s.render(g)
62     } else {
63         // Destroy splashscreen physics lines and reposition player
64         if (firstRun) {
65             l.destroy()
66             l2.destroy()
67             playerMachine.setPos(camX, camY)
68         }
69         // Load screen: reset camera and render the file picker
70         if (currentMode == "load") {
71             camX = 1920 / 2
72             camY = 540
73             cam.position.set(camX, camY, 0)
74             cam.update()
75             s1.render(g)
76         } else {
77             // Main game rendering
78             firstRun = false
79             g.clear(Color.WHITE)
80             g.setColor(Color.BLACK)
81
82             cam = g.getCamera
83             PhysicsWorld.updatePhysics()

```

```

84 playerMachine.update()
85
86 // Camera follows the player in play mode, otherwise stays at manually-set position
87 if (currentMode == "play") {
88     cam.position.set(playerMachine.posX, playerMachine.posY, 0)
89 } else {
90     cam.position.set(camX, camY, 0)
91 }
92 cam.update()
93
94 playerMachine.draw(g)
95
96 // Freeze the player outside of play mode
97 if (currentMode != "play") {
98     playerMachine.sleep()
99     playerMachine.angle = 0
100 }
101
102 // Use camera or player position as the drawing reference point
103 if (currentMode == "play") drawZoneX = camX else drawZoneX = playerMachine.posX.toInt
104 if (currentMode != "play") drawZoneY = camY else drawZoneY = playerMachine.posY.toInt
105
106 lineMachine.drawLines(g, modesMachine.currentMode(), modesMachine.getDrawMode(), drawZoneX, drawZoneY)
107 freeMachine.drawFreeLines(g, modesMachine.currentMode(), modesMachine.getDrawMode(), drawZoneX, drawZoneY)
108
109 lastMode = currentMode
110 currentMode = modesMachine.currentMode()
111 musicMode = modesMachine.getMusicMode()
112
113
114 // Draw the UI menu and mode-specific overlays
115 g.resetCamera()
116 modesMachine.drawModesMenu(g, g.getScreenHeight(), 0)
117 if (currentMode == "mop") {
118     uSure.drawWindow(g)
119 }
120 if (isSaving && currentMode != "mop" &&
121     (System.currentTimeMillis() - saveTime) < 5000) {
122     saveWin.drawWindow(g, savefile)
123 }
124
125 val oldColor = g.sbGetColor()
126 g.setColor(Color.BLACK)
127 // Show coordinates (player or camera position)
128 if (currentMode == "play") {
129     g.drawString(5, 50, s"X : ${playerMachine.posX.toInt}")
130     g.drawString(5, 35, s"Y : ${playerMachine.posY.toInt}")
131 } else {
132     g.drawString(5, 50, s"X : $camX")
133     g.drawString(5, 35, s"Y : $camY")
134 }
135 g.setColor(oldColor)
136 g.drawFPS(Color.BLACK)
137 g.drawSchoolLogo()
138 }
139 }
140 }
141
142 // Handles mouse clicks
143 override def onClick(x: Int, y: Int, button: Int): Unit = {
144     currentX = x
145     currentY = y
146
147     lastMouseClicked = button //On enregistre si click droit ou gache pour prevent le drag sur clic droit
148
149     super.onClick(x, y, button)
150
151     if (button == Input.Buttons.LEFT) {
152         onMenuClick = modesMachine.onMenuClick(x, y)
153         currentMode = modesMachine.currentMode()
154     }

```

```

155 //Maintenant que le monde bouge il faut calculer différemment l'emplacement de la souris
156 // x et y du clic sont relatif à la fenêtre et non au monde, le pixel haut gauche sera toujours à (0,0)
157 // indépendamment du mouvement dans le monde derrière
158 // il faut donc calculer par nous même cette position
159
160 // Convert screen coordinates to world coordinates (accounting for camera offset)
161 val inWorldClicX: Int = x + (camX - 960)
162 val inWorldClicY: Int = y + (camY - 540)
163
164 currentMode match {
165     case "free" => {
166         if (!onMenuClick) {
167             freeMachine.onClick("LEFT", inWorldClicX, inWorldClicY)
168         } else {
169             playerMachine.setPos(960, 900)
170         }
171     }
172     case "lines" => {
173         if (!onMenuClick) {
174             lineMachine.onClick("LEFT", inWorldClicX, inWorldClicY)
175         } else {
176             playerMachine.setPos(960, 900)
177         }
178     }
179     case "play" => {
180         if (lastMode != "play") {
181             playerMachine.setPos(960, 900)
182             playerMachine.awake()
183         }
184     }
185     case "eraser" => {
186         playerMachine.setPos(960, 900)
187         iAmClicked = true
188         freeMachine.clean(inWorldClicX, inWorldClicY)
189         lineMachine.clean(inWorldClicX, inWorldClicY)
190     }
191     case "mop" => {
192         uSure.onClick(x, y)
193         wasMopping = true
194         if (uSure.getAnswer == "yes") {
195             freeMachine.mop()
196             lineMachine.mop()
197             modesMachine.modeSwitcher("lines")
198             lineMachine.onClick("LEFT", -100000, -100000)
199             lineMachine.endPoint.set(0f, 0f)
200             lineMachine.startPoint.set(0f, 0f)
201         } else if (uSure.getAnswer == "no") {
202             modesMachine.modeSwitcher("lines")
203             lineMachine.onClick("LEFT", -100000, -100000)
204             lineMachine.endPoint.set(0f, 0f)
205             lineMachine.startPoint.set(0f, 0f)
206         }
207     }
208     case "return" => {
209         camX = playerMachine.posX.toInt
210         camY = playerMachine.posY.toInt
211         modesMachine.modeSwitcher("lines")
212     }
213     case "save" => {
214         isSaving = true
215         // Generate a unique filename using the current date/time
216         val c: Calendar = Calendar.getInstance()
217         val fn: String = "save_" +
218             s"${c.get(Calendar.DAY_OF_YEAR)}" +
219             s"${c.get(Calendar.HOUR_OF_DAY)}" +
220             s"${c.get(Calendar.MINUTE)}" +
221             s"${c.get(Calendar.SECOND)}"
222
223         lineMachine.save(fn)

```

```

224     freeMachine.save(fn)
225     savefile = fn + ".csv"
226     saveTime = System.currentTimeMillis()
227     println(s"Game saved : $fn.csv")
228     modesMachine.modeSwitcher("lines")
229 }
230 case "load" => {
231     s1.getActiveScreen match {
232         case lw: LoadWindow => {
233             lw.onClick(x, y)
234             //println(lw.getAnswer())
235             if (lw.getAnswer() == "load") {
236                 lineMachine.mop()
237                 freeMachine.mop()
238                 lineMachine.load(lw.getSelection())
239                 modesMachine.modeSwitcher("lines")
240                 lineMachine.onClick("LEFT", -100000, -100000)
241                 lineMachine.endPoint.set(0f, 0f)
242                 lineMachine.startPoint.set(0f, 0f)
243             }
244             if (lw.getAnswer() == "cancel") {
245                 modesMachine.modeSwitcher("lines")
246                 lineMachine.onClick("LEFT", -100000, -100000)
247                 lineMachine.endPoint.set(0f, 0f)
248                 lineMachine.startPoint.set(0f, 0f)
249             }
250         }
251         case _ =>
252     }
253 }
254 case _ => {
255     playerMachine.setPos(960, 900)
256 }
257 }
258
259 } else {
260     // Right-click starts camera drag / in play mode switches to edit mode first
261     RDragX = x
262     RDragY = y
263     if (currentMode == "play") {
264         modesMachine.modeSwitcher("lines")
265         playerMachine.setPos(960, 900)
266         camX = playerMachine.posX.toInt
267         camY = playerMachine.posY.toInt
268     }
269 }
270
271 }
272
273
274 // Handles mouse drag: drawing input or camera panning
275 override def onDrag(x: Int, y: Int): Unit = {
276     currentX = x
277     currentY = y
278     val physicMode: String = modesMachine.getDrawMode()
279
280     val inWorldClicX: Int = x + (camX - 960)
281     val inWorldClicY: Int = y + (camY - 540)
282
283     if (!onMenuClick && lastMouseClicked == Input.Buttons.LEFT) {
284         currentMode match {
285             case "free" => freeMachine.onDrag(inWorldClicX, inWorldClicY, physicMode)
286             case "lines" => if (!wasMopping) lineMachine.onDrag(inWorldClicX, inWorldClicY)
287             case "play" => {}
288             case "eraser" => {
289                 freeMachine.clean(inWorldClicX, inWorldClicY)
290                 lineMachine.clean(inWorldClicX, inWorldClicY)
291             }
292             case _ => {}
293         }
294     }

```

```

294 // Right-click drag: moves the camera
295 } else if (lastMouseClicked == Input.Buttons.RIGHT && currentMode != "load") {
296     camX = camX - (x - RDragX)
297     camY = camY - (y - RDragY)
298     RDragX = x
299     RDragY = y
300     cam.position.set(camX, camY, 0)
301     cam.update()
302 }
303
304 }
305
306 // Handles mouse release: finalises the current drawing action
307 override def onRelease(x: Int, y: Int, button: Int): Unit = {
308     currentX = x
309     currentY = y
310     val physicMode: String = modesMachine.getDrawMode()
311
312     super.onRelease(x, y, button)
313
314     val inWorldClicX: Int = x + (camX - 960)
315     val inWorldClicY: Int = y + (camY - 540)
316
317     if (!onMenuClick) {
318         super.onRelease(x, y, button)
319         lineMachine.firstRun = false
320         freeMachine.firstRun = false
321
322         if (button == Input.Buttons.LEFT) {
323             currentMode match {
324                 case "free" => freeMachine.onRelease("LEFT", inWorldClicX, inWorldClicY)
325                 case "lines" => if (!wasMopping) {
326                     lineMachine.onRelease("LEFT", inWorldClicX, inWorldClicY, physicMode)
327                 } else {
328                     wasMopping = false
329                 }
330                 case "play" => {}
331                 case "eraser" => iAmClicked = false
332                 case _ => {}
333             }
334             //println("Left button released")
335         } else {
336             //println("Right button released")
337         }
338     }
339 }
340 }

```

Listing 3 - Classe principale du jeu - Game.scala

```

1 import ch.hevs.gdx2d.components.bitmaps.BitmapImage
2 import ch.hevs.gdx2d.components.physics.primitives.PhysicsCircle
3 import ch.hevs.gdx2d.components.physics.utils.PhysicsConstants
4 import ch.hevs.gdx2d.lib.GdxGraphics
5 import ch.hevs.gdx2d.lib.interfaces.DrawableObject
6 import ch.hevs.gdx2d.lib.physics.AbstractPhysicsObject
7 import com.badlogic.gdx.math.Vector2
8
9 // Player character: a physics circle with a sledge image, responds to collisions
10 class MudryMachine(name: String, position: Vector2, radius: Float, density: Float, restitution: Float, friction: Float)
11     extends PhysicsCircle(name, position, radius, density, restitution, friction) with DrawableObject {
12
13     var firstRun = true
14     var posX: Float = 960
15     var posY: Float = 900
16     var angle: Float = 0
17     var img: BitmapImage = _
18     private var rider: BitmapImage = _
19     private var imgDown: BitmapImage = _

```

```

20 private var selectedImage: BitmapImage = _
21 private var lastCollision = 0.5f
22
23 // Teleport the player to a world position
24 def setPos(x: Int, y: Int): Unit = {
25     val muBody = this.getBody
26     val positionMeters = new Vector2(x, y).scl(PhysicsConstants.P2M)
27     muBody.setTransform(positionMeters, 0)
28     this.setBodyLinearVelocity(new Vector2(0, 0))
29     posX = x
30     posY = y
31 }
32
33 // Enable physics on the player
34 def awake(): Unit = {
35     this.setBodyAwake(true)
36     this.enableCollisionListener()
37 }
38
39 // Freeze the player (disables physics)
40 def sleep(): Unit = {
41     this.setBodyAwake(false)
42 }
43
44 // Draw the player
45 def draw(g: GdxGraphics): Unit = {
46     loadImages()
47
48     if (math.abs(this.angle) > 30) {
49         selectedImage = imgDown
50     } else {
51         selectedImage = img
52     }
53
54     g.drawTransformedPicture(posX, posY, this.angle, 0.1f, selectedImage)
55     g.drawTransformedPicture(posX, posY, this.angle / 2, 0.1f, rider)
56
57 }
58
59 def loadImages(): Unit = {
60     if (firstRun) {
61         if (img == null) {
62             rider = new BitmapImage("./icons/no-sled-mud.png")
63             img = new BitmapImage("./icons/sledge.png")
64             imgDown = new BitmapImage("./icons/sledge.png")
65             selectedImage = img
66         }
67         firstRun = false
68     }
69 }
70
71 // Sync stored position with the physics body position
72 def update(): Unit = {
73     posX = this.getBodyPosition.x
74     posY = this.getBodyPosition.y
75 }
76
77 // Computes the player's tilt angle on collision
78 override def collision(other: AbstractPhysicsObject, energy: Float): Unit = {
79     //this.angle = this.angle + 1 //prout <----- (commentaire relique de St-Mui)
80     angle = (other.getBodyAngle * 180 / math.Pi).toFloat
81
82     if (angle <= -120) {
83         angle += 180
84     }
85     if (angle >= 120) {
86         angle -= 180
87     }
88
89     lastCollision = 1.0f

```

```

90 }
91 }

```

Listing 4 - Personnage physique du jeu - MudryMachine.scala

```

1 import java.io.File
2 import javax.sound.sampled.AudioSystem
3
4 // Manages background music, switching tracks based on the current game mode
5
6 class MusicPlayer() {
7     private val clip = AudioSystem.getClip()
8     private var lastmode: String = ""
9     private var lastMusicmode: String = ""
10
11     def play(currentMode: String, musicMode: String): Unit = {
12         if (musicMode == "musicmute" && clip.isActive) { // Si la musique est coupée en jeu, fermer le clip
13             lastMusicmode = "musicmute"
14             if (clip.isOpen) {
15                 clip.stop()
16                 clip.close()
17             }
18         }
19
20         if (currentMode != lastmode && musicMode != "musicmute" ||
21             lastMusicmode != musicMode && musicMode != "musicmute" ||
22             !clip.isActive && musicMode != "musicmute") {
23             currentMode match { // Else, we play the music based on the game mode
24                 case "play" => {
25                     if (!clip.isActive || lastmode != "play") { //if the mode play is selected, play the music "play.wav"
26                         if (lastmode != currentMode) {
27                             if (clip.isOpen) {
28                                 clip.stop()
29                                 clip.close()
30                             }
31                             lastmode = currentMode
32                         }
33
34                         val musicfile = new File(s"./music/play.wav")
35                         val audio = AudioSystem.getAudioInputStream(musicfile)
36                         clip.open(audio)
37                         clip.loop(-1)
38                         clip.start()
39                     }
40                 }
41                 case ("free" | "lines" | "eraser" | "mop") => { //if an edition mode is selected, play the music "edit.wav"
42                     if (lastmode != "free" && lastmode != "lines" && lastmode != "mop" && lastmode != "eraser") {
43                         if (clip.isOpen) {
44                             clip.stop()
45                             clip.close()
46                         }
47                     }
48                     if (!clip.isActive &&
49                         (lastmode != "free" || lastmode != "lines" || lastmode != "mop" || lastmode != "eraser")) {
50                         if (lastmode != currentMode) {
51                             if (clip.isOpen) {
52                                 clip.stop()
53                                 clip.close()
54                             }
55                             lastmode = currentMode
56                         }
57                     }
58
59                     val musicfile = new File(s"./music/edit.wav")
60                     val audio = AudioSystem.getAudioInputStream(musicfile)
61                     clip.open(audio)
62                     clip.loop(-1)
63                     clip.start()
64                 }
65             }
66         }
67     }
68 }

```



```

65     case _ => { //else stop everything
66         lastmode = currentMode
67         if (clip.isOpen) {
68             clip.stop()
69             clip.close()
70         }
71     }
72 }
73 }
74 lastmode = currentMode
75 }
76 }

```

Listing 5 - Gestion de la musique - MusicPlayer.scala

9.3 – Systèmes de dessin

```

1  import ch.hevs.gdx2d.lib.GdxGraphics
2  import com.badlogic.gdx.graphics.Color
3  import com.badlogic.gdx.math.Vector2
4
5  import java.io.{FileOutputStream, PrintWriter}
6  import scala.collection.mutable.ArrayBuffer
7  import scala.io.Source
8
9  // Manages drawing, erasing, saving, and loading physics and decorative lines
10
11
12  class LineDrawMachine {
13      val calc: Calculator = new Calculator
14      private val LineArray: ArrayBuffer[Line] = ArrayBuffer.empty
15      var startPoint: Vector2 = new Vector2()
16      var endPoint: Vector2 = new Vector2()
17      var lastEndPoint: Vector2 = new Vector2()
18      var isMousePressed: Boolean = false
19      var firstRun: Boolean = true
20      var cursorLoc = new Vector2()
21
22      ArrayEmptyFix()
23
24      def drawLines(g: GdxGraphics, cm: String, dm: String, camX: Int, camY: Int): Unit = {
25          if (dm != "decoration") g.setColor(Color.BLACK) else g.setColor(Color.BLUE)
26          if (endPoint.x != 0.0f && endPoint.y != 0.0f) g.drawLine(startPoint.x, startPoint.y, endPoint.x, endPoint.y)
27          for (line <- LineArray) {
28              if (line.isInstanceOf[DecoLine] && cm != "play") {
29                  line.color = Color.BLUE
30              } else {
31                  line.color = Color.BLACK
32              }
33              // Only draw lines within the visible area (plus a margin), or the dummy placeholder
34              if (line.p1x >= (camX - 2000) && line.p1x <= (camX + 2000) ||
35                  line.p2x >= (camX - 2000) && line.p2x <= (camX + 2000) ||
36                  line.p1y >= (camY - 1500) && line.p1y <= (camY + 1500) ||
37                  line.p2y >= (camY - 1500) && line.p2y <= (camY + 1500) ||
38                  line.p1x == -10000 && line.p2y == -10000) {
39                  line.draw(g)
40              }
41          }
42      }
43
44      def onClick(mode: String, x: Int, y: Int): Unit = {
45          mode match {
46              case "RIGHT" => {}
47              case "LEFT" => {
48                  if (lastEndPoint.x == endPoint.x && lastEndPoint.y == endPoint.y) endPoint.set(x, y)
49                  startPoint.set(x, y)
50                  isMousePressed = true
51              }
52          }
53      }
54  }

```

```

53 }
54
55 def onDrag(x: Int, y: Int): Unit = {
56   endPoint.set(x, y)
57 }
58
59 def onRelease(mode: String, x: Int, y: Int, dm: String): Unit = {
60   mode match {
61     case "RIGHT" => {}
62     case "LEFT" => {
63       isMousePressed = false
64       endPoint.set(x, y)
65
66       dm match {
67         case "physic" => {
68           val l1 = PhysicLine(startPoint.x, startPoint.y, endPoint.x, endPoint.y)
69           LineArray.addOne(l1)
70           lastEndPoint.set(endPoint.x, endPoint.y)
71         }
72         case "decoration" => {
73           val l1 = DecoLine(startPoint.x, startPoint.y, endPoint.x, endPoint.y)
74           LineArray.addOne(l1)
75           lastEndPoint.set(endPoint.x, endPoint.y)
76         }
77       }
78     }
79   }
80 }
81
82 def clean(x: Int, y: Int): Unit = {
83   ArrayEmptyFix()
84   cursorLoc.set(x, y)
85   val toRemove: ArrayBuffer[Line] = ArrayBuffer.empty
86
87   for (line <- LineArray) {
88     if (calc.isPointInLine(line, cursorLoc)) {
89       if (!toRemove.contains(line))
90         toRemove.addOne(line)
91     }
92   }
93   for (line <- toRemove) {
94     LineArray -= line
95     line.destroy()
96   }
97   endPoint.set(0f, 0f)
98   startPoint.set(0f, 0f)
99   ArrayEmptyFix()
100 }
101
102 def mop(): Unit = {
103   for (line <- LineArray) {
104     line.destroy()
105   }
106   LineArray.clear()
107   ArrayEmptyFix()
108 }
109
110 // HACK: creates a dummy line so the physics world doesn't break when empty
111 private def ArrayEmptyFix(): Unit = {
112   if (LineArray.isEmpty) {
113     val l1 = PhysicLine(-10000, -10000, -10000, -10000)
114     LineArray.addOne(l1)
115   }
116 }
117
118 def save(filename: String): Unit = {
119   val pw = new PrintWriter(
120     new FileOutputStream(s"./saves/$filename.csv", true)
121   )
122   // Save format: TypeLine,p1x,p1y,p2x,p2y per line

```

```

123 for (line <- LineArray) {
124     pw.println(line.getClass.getSimpleName + "," +
125         line.p1x + "," + line.p1y + "," + line.p2x + "," + line.p2y)
126 }
127 pw.close()
128 }
129
130 def load(fp: String): Unit = {
131     val src = Source.fromFile(s"./saves/$fp")
132     val lines = src.getLines().toArray
133     for (line <- lines) {
134         val a = line.split(",")
135         if (line.contains("PhysicLine")) {
136             LineArray.addOne(PhysicLine(a(1).toFloat, a(2).toFloat, a(3).toFloat, a(4).toFloat))
137         }
138         if (line.contains("DecoLine")) {
139             LineArray.addOne(DecoLine(a(1).toFloat, a(2).toFloat, a(3).toFloat, a(4).toFloat))
140         }
141     }
142     src.close()
143 }
144 }

```

Listing 6 - Dessin de lignes segmentées - LineDrawMachine.scala

```

1 import ch.hevs.gdx2d.lib.GdxGraphics
2 import com.badlogic.gdx.graphics.Color
3 import com.badlogic.gdx.math.Vector2
4 import typesLibrary.Free
5
6 import java.io.{FileOutputStream, PrintWriter}
7 import scala.collection.mutable.ArrayBuffer
8
9 // Manages free-hand drawing (sequences of short segments), erasing, and saving
10
11 class FreeDrawMachine {
12     val calc: Calculator = new Calculator
13     private val FreeArray: ArrayBuffer[Free] = ArrayBuffer.empty
14     var startPoint: Vector2 = new Vector2()
15     var endPoint: Vector2 = new Vector2()
16     var lastEndPoint: Vector2 = new Vector2()
17     var isMousePressed: Boolean = false
18     private var FreeCnt: Int = -1
19     var firstRun: Boolean = true
20     var cursorLoc = new Vector2()
21
22     ArrayEmptyFix()
23
24     // IF THE LINE IS CLOSE TO THE VISIBLE AREA OF THE CAMERA, IT DRAWS IT
25     def drawFreeLines(g: GdxGraphics, cm: String, dm: String, camX: Int, camY: Int): Unit = {
26         if (dm != "decoration") g.setColor(Color.BLACK) else g.setColor(Color.BLUE)
27         if (endPoint.x != 0.0f && endPoint.y != 0.0f) g.drawLine(startPoint.x, startPoint.y, endPoint.x, endPoint.y)
28         for (free <- FreeArray) {
29             for (segment <- free) {
30                 if (segment.isInstanceOf[DecoLine] && cm != "play") {
31                     segment.color = Color.BLUE
32                 } else {
33                     segment.color = Color.BLACK
34                 }
35                 // Only draw segments within the visible area (plus a margin), or the dummy placeholder
36                 if (segment.p1x >= (camX - 2000) && segment.p1x <= (camX + 2000) ||
37                     segment.p2x >= (camX - 2000) && segment.p2x <= (camX + 2000) ||
38                     segment.p1y >= (camY - 1500) && segment.p1y <= (camY + 1500) ||
39                     segment.p2y >= (camY - 1500) && segment.p2y <= (camY + 1500) ||
40                     segment.p1x == -10000 && segment.p2y == -10000) {
41                     segment.draw(g)
42                 }
43             }
44         }
45     }
46 }

```

```

45 }
46 }
47
48 def onClick(mode: String, x: Int, y: Int): Unit = {
49   mode match {
50     case "RIGHT" => {}
51     case "LEFT" => {
52       val f1 = new Free
53       FreeArray.addOne(f1)
54       FreeCnt += 1
55       if (lastEndPoint.x == endPoint.x && lastEndPoint.y == endPoint.y) endPoint.set(x, y)
56       startPoint.set(x, y)
57       isMousePressed = true
58     }
59   }
60 }
61
62 // Add a new segment from the previous point to the current mouse position
63 def onDrag(x: Int, y: Int, dm: String): Unit = {
64   endPoint.set(x, y)
65   dm match {
66     case "physic" => {
67       val seg1 = PhysicLine(startPoint.x, startPoint.y, endPoint.x, endPoint.y)
68       lastEndPoint.set(endPoint.x, endPoint.y)
69       FreeArray(FreeCnt).addOne(seg1)
70       startPoint.set(endPoint.x, endPoint.y)
71     }
72     case "decoration" => {
73       val seg1 = DecoLine(startPoint.x, startPoint.y, endPoint.x, endPoint.y)
74       lastEndPoint.set(endPoint.x, endPoint.y)
75       FreeArray(FreeCnt).addOne(seg1)
76       startPoint.set(endPoint.x, endPoint.y)
77     }
78   }
79 }
80
81 def onRelease(mode: String, x: Int, y: Int): Unit = {
82   mode match {
83     case "RIGHT" => {}
84     case "LEFT" => isMousePressed = false
85   }
86 }
87
88 def clean(x: Int, y: Int): Unit = {
89   ArrayEmptyFix()
90   cursorLoc.set(x, y)
91   val toRemove: ArrayBuffer[Line] = ArrayBuffer.empty
92
93   for (free <- FreeArray) {
94     for (segment <- free) {
95       if (calc.isPointInSegment(segment, cursorLoc)) {
96         if (!toRemove.contains(segment))
97           toRemove.addOne(segment)
98       }
99     }
100   }
101   for (free <- FreeArray) {
102     for (segment <- toRemove) {
103       if (free.contains(segment)) {
104         free -= segment
105         segment.destroy()
106       }
107     }
108   }
109 }
110
111 def mop(): Unit = {
112   for (free <- FreeArray) {
113     for (segmt <- free) {
114       segmt.destroy()

```

```

115     }
116   }
117   for (free <- FreeArray) {
118     free.clear()
119   }
120   ArrayEmptyFix()
121 }
122
123 // HACK: creates a dummy segment so the physics world doesn't break when empty
124 private def ArrayEmptyFix(): Unit = {
125   if (FreeArray.isEmpty) {
126     val l1 = PhysicLine(-10000, -10000, -10000, -10000)
127     FreeArray.addOne(new ArrayBuffer[Line]().addOne(l1))
128   }
129 }
130
131 def save(filename: String): Unit = {
132   val pw = new PrintWriter(
133     new FileOutputStream(s"./saves/$filename.csv", true)
134   )
135   // Save format: TypeLine,p1x,p1y,p2x,p2y per line segment
136   for (free <- FreeArray) {
137     for (sgmt <- free) {
138       pw.println(sgmt.getClass.getSimpleName + "," +
139         sgmt.p1x + "," + sgmt.p1y + "," + sgmt.p2x + "," + sgmt.p2y)
140     }
141   }
142   pw.close()
143 }
144
145 }

```

Listing 7 - Dessin de lignes libres - FreeDrawMachine.scala

```

1 import com.badlogic.gdx.math.Vector2
2
3 // Geometry calculations for the eraser: point-to-segment distance and hit testing
4
5 class Calculator {
6   private val tolerance = 8 //Mouse tolerance, nbr of pixels around the mouse
7   private val vDir = new Vector2() //Vecteur Directeur
8   private val vXP = new Vector2() // Vecteur entre le point X et le point P (P = Mouse location)
9
10  //The logic is the same as a Segment, but to make it consistant, we added this fonction that equals isPointInSegment
11  def isPointInLine(sgmt: Line, point: Vector2): Boolean = {
12    isPointInSegment(sgmt, point)
13  }
14
15  //The mouse is in the segment : if the distance between it is in tolerance and if it is on the said segment
16  def isPointInSegment(sgmt: Line, point: Vector2): Boolean = {
17    if (distanceToSegment(sgmt, point) <= tolerance) {
18      if (isPointInBoundaries(sgmt, point)) {
19        return true
20      }
21    }
22    false
23  }
24
25  //Returns true if the point is part of a said segment
26  private def isPointInBoundaries(sgmt: Line, point: Vector2): Boolean = {
27    val minX = math.min(sgmt.p1x, sgmt.p2x) - tolerance
28    val maxX = math.max(sgmt.p1x, sgmt.p2x) + tolerance
29    val minY = math.min(sgmt.p1y, sgmt.p2y) - tolerance
30    val maxY = math.max(sgmt.p1y, sgmt.p2y) + tolerance
31
32    if (minX <= point.x && point.x <= maxX) {
33      if (minY <= point.y && point.y <= maxY) {
34        return true
35      }
36    }
37  }

```

```

36     }
37     false
38 }
39
40 //Returns the distance between the mouse and the vector
41 private def distanceToSegment(sgmt: Line, point: Vector2): Double = {
42     vDir.set((sgmt.p2x - sgmt.p1x), (sgmt.p2y - sgmt.p1y))
43     vXP.set((point.x - sgmt.p1x), (point.y - sgmt.p1y))
44     val vectorProduct = vDir.x * vXP.y - vDir.y * vXP.x
45     var vDirNorm = math.sqrt(vDir.x * vDir.x + vDir.y * vDir.y)
46     if (vDirNorm == 0) vDirNorm = Double.MinPositiveValue //Removing the issue of dividing by zero
47     val distance = (math.abs(vectorProduct) / vDirNorm)
48     distance
49 }
50 }

```

Listing 8 - Calculs géométriques - Calculator.scala

9.4 – Types de lignes

```

1 import ch.hevs.gdx2d.lib.GdxGraphics
2 import com.badlogic.gdx.graphics.Color
3
4 // Common interface for all line types: physics lines and decorative lines
5 trait Line {
6     var color: Color
7
8     def p1x: Float
9
10    def p1y: Float
11
12    def p2x: Float
13
14    def p2y: Float
15
16    def destroy() {}
17
18    def draw(g: GdxGraphics) {}
19
20 }

```

Listing 9 - Trait de base pour les lignes - Line.scala

```

1 import ch.hevs.gdx2d.components.physics.primitives.PhysicsStaticLine
2 import ch.hevs.gdx2d.lib.GdxGraphics
3 import ch.hevs.gdx2d.lib.physics.AbstractPhysicsObject
4 import com.badlogic.gdx.graphics.Color
5 import com.badlogic.gdx.math.Vector2
6
7 // A physics-enabled line: interacts with the player
8 case class PhysicLine(p1x: Float, p1y: Float, p2x: Float, p2y: Float)
9 extends PhysicsStaticLine("a line", new Vector2(p1x, p1y), new Vector2(p2x, p2y), 0.1f, 0.5f, 0.4f) with Line {
10     var color: Color = Color.BLACK
11
12     override def collision(other: AbstractPhysicsObject, energy: Float): Unit = {
13     }
14
15     override def destroy(): Unit = {
16         super[PhysicsStaticLine].destroy()
17     }
18
19     override def draw(g: GdxGraphics): Unit = {
20         g.setColor(color)
21         g.drawLine(p1x, p1y, p2x, p2y)
22     }
23 }

```

Listing 10 - Ligne physique - PhysicLine.scala

```

1 import ch.hevs.gdx2d.lib.GdxGraphics
2 import com.badlogic.gdx.graphics.Color
3
4 // A decorative line (no physics, rendered in blue when editing and in black when playing)
5
6 case class DecoLine(p1x: Float, p1y: Float, p2x: Float, p2y: Float)
7   extends Line {
8     var color: Color = Color.BLACK
9
10    override def destroy(): Unit = {
11      super.destroy()
12    }
13
14    override def draw(g: GdxGraphics): Unit = {
15      g.setColor(color)
16      g.drawLine(p1x, p1y, p2x, p2y)
17    }
18  }

```

Listing 11 - Ligne de décoration - DecoLine.scala

9.5 – Fenêtres UI

```

1 import ch.hevs.gdx2d.components.bitmaps.BitmapImage
2 import ch.hevs.gdx2d.components.screen_management.RenderingScreen
3 import ch.hevs.gdx2d.lib.GdxGraphics
4 import ch.hevs.gdx2d.lib.physics.PhysicsWorld
5 import com.badlogic.gdx.Gdx
6 import com.badlogic.gdx.graphics.Color
7 import com.badlogic.gdx.graphics.g2d.BitmapFont
8 import com.badlogic.gdx.graphics.g2d.freetype.FreeTypeFontGenerator
9 import com.badlogic.gdx.graphics.g2d.freetype.FreeTypeFontGenerator.FreeTypeFontParameter
10 import com.badlogic.gdx.math.Vector2
11
12
13 // Animated splash screen shown at startup with title and player image
14 class SplashScreenWindow extends RenderingScreen {
15
16   private val param: FreeTypeFontParameter = new FreeTypeFontParameter
17   param.size = 50
18   param.color = Color.BLACK
19   private val customFont: BitmapFont =
20     new FreeTypeFontGenerator(Gdx.files.local("font/Acme-Regular.ttf")).generateFont(param)
21   private var mM: MudryMachine = _
22   var c = 0
23   private var imgBitmap: BitmapImage = _
24
25   // Load splash image and create a dummy player for the animation
26   override def onInit(): Unit = {
27     imgBitmap = new BitmapImage("icons/mudry2.png")
28     mM = new MudryMachine("rider2", new Vector2(960, 900), 10, 1f, 0.65f, 0.6f)
29     mM.loadImages()
30     mM.setPos(100, 1200)
31   }
32
33   // Render the splash: animated title text + falling player image + school logo
34   override def onGraphicRender(g: GdxGraphics): Unit = {
35     g.clear(Color.WHITE)
36     g.setColor(Color.BLACK)
37     g.drawStringCentered(((g.getScreenHeight / 2) + c).toFloat, "MudRYder", customFont)
38     c += 2
39     g.setColor(Color.WHITE)
40
41     mM.awake()
42     mM.update()
43     PhysicsWorld.updatePhysics()
44
45     g.drawTransformedPicture(mM.posX, mM.posY, 0, 0.7f, imgBitmap)
46     g.drawSchoolLogo()

```

```

47 }
48
49 override def dispose(): Unit = {
50   imgBitmap.dispose()
51   mM.destroy()
52 }
53
54 }

```

Listing 12 - Écran de démarrage - SplashScreenWindow.scala

```

1  import ch.hevs.gdx2d.components.screen_management.RenderingScreen
2  import ch.hevs.gdx2d.lib.GdxGraphics
3  import com.badlogic.gdx.Gdx
4  import com.badlogic.gdx.files.FileHandle
5  import com.badlogic.gdx.graphics.Color
6  import com.badlogic.gdx.math.Vector2
7
8  import scala.collection.mutable.ListBuffer
9
10
11 // Full-screen file selection screen for loading saved levels
12 class LoadWindow extends RenderingScreen {
13   var files: Array[FileHandle] = _
14   private val filesList: ListBuffer[String] = ListBuffer.empty
15
16   private var cancelButtonCo: Vector2 = new Vector2(0, 0)
17   private var loadButtonCo: Vector2 = new Vector2(0, 0)
18   var answer: String = ""
19   private var selectedSaveIndice: Int = -1
20
21   // List all .csv save files from the saves directory
22   override def onInit(): Unit = {
23     files = Gdx.files.local("saves").list()
24     files.foreach { f =>
25       if (f.name().contains(".csv"))
26         filesList.addOne(f.name())
27     }
28   }
29
30   // Draw the file list and Cancel/Load buttons
31   override def onGraphicRender(g: GdxGraphics): Unit = {
32     g.clear(Color.WHITE)
33     g.setColor(Color.BLACK)
34     g.drawStringCentered(800, "Files available : ")
35     var c = 40
36
37     for (filename <- filesList) {
38       g.drawStringCentered(800 - c, filename)
39
40       if (selectedSaveIndice != -1) {
41         if (filename == filesList(selectedSaveIndice)) {
42           g.setColor(Color.BLUE)
43           g.drawRectangle(960, (795 - c), 160, 20, 0)
44           g.setColor(Color.BLACK)
45         }
46       }
47
48       c += 30
49     }
50
51     if (cancelButtonCo.x == 0 && cancelButtonCo.y == 0) {
52       cancelButtonCo = new Vector2((g.getScreenWidth / 2 - 80), (750 - c))
53     }
54     if (loadButtonCo.x == 0 && loadButtonCo.y == 0) {
55       loadButtonCo = new Vector2((g.getScreenWidth / 2 + 80), (750 - c))
56     }
57
58     g.drawRectangle(cancelButtonCo.x, cancelButtonCo.y, 100, 30, 0)
59

```



```

60 g.drawString((g.getScreenWidth / 2 - 105), 755 - c, "Cancel")
61 g.drawRectangle(loadButtonCo.x, loadButtonCo.y, 100, 30, 0)
62 g.drawString((g.getScreenWidth / 2 + 65), 755 - c, "Load")
63 }
64
65 // Handle clicks: selecting a file, or pressing Cancel/Load
66 def onClick(x: Int, y: Int): Unit = {
67   if (x >= cancelButtonCo.x - 50 && x <= cancelButtonCo.x + 50 &&
68       y >= cancelButtonCo.y - 15 && y <= cancelButtonCo.y + 15) {
69     answer = "cancel"
70   } else if (x >= loadButtonCo.x - 50 && x <= loadButtonCo.x + 50 &&
71       y >= loadButtonCo.y - 15 && y <= loadButtonCo.y + 15) {
72     answer = "load"
73   } else {
74     answer = ""
75   }
76
77   for (c <- 0 until (filesList.length * 30) by 30) {
78     if (x >= 880 && x <= 1040 &&
79         y >= 745 - c && y <= 765 - c) {
80       selectedSaveIndice = c / 30
81     }
82   }
83
84 }
85
86 // Return the filename the user selected
87 def getSelection(): String = {
88   filesList(selectedSaveIndice)
89 }
90
91
92 // Return the user's button choice ("load" or "cancel")
93 def getAnswer(): String = {
94   answer
95 }
96
97 }

```

Listing 13 - Fenêtre de chargement - LoadWindow.scala

```

1 import ch.hevs.gdx2d.components.graphics.Polygon
2 import ch.hevs.gdx2d.lib.GdxGraphics
3 import com.badlogic.gdx.graphics.Color
4 import com.badlogic.gdx.math.Vector2
5
6 // Temporary notification overlay shown after saving, displaying the filename
7 class SavingWindow {
8
9   val firstWindowPoint: Vector2 = new Vector2(280, 960)
10   val windowHeight: Int = 55
11   val windowWidth: Int = 165
12
13   val windowCo: Array[Vector2] = Array(
14     firstWindowPoint,
15     new Vector2(firstWindowPoint.x, firstWindowPoint.y + windowHeight),
16     new Vector2(firstWindowPoint.x + windowWidth, firstWindowPoint.y + windowHeight),
17     new Vector2(firstWindowPoint.x + windowWidth, firstWindowPoint.y)
18   )
19
20   def drawWindow(g: GdxGraphics, fn: String): Unit = {
21     val oldColor = g.getColor()
22     g.setColor(Color.BLACK)
23     g.drawFilledPolygon(new Polygon(windowCo), Color.WHITE)
24     g.drawPolygon(new Polygon(windowCo))
25     g.drawString(firstWindowPoint.x + 33, firstWindowPoint.y + windowHeight - 10, "Saving File...")
26     g.drawString(firstWindowPoint.x + 10, firstWindowPoint.y + windowHeight - 33, fn)
27     g.setColor(oldColor)
28   }
29 }

```

30 }

Listing 14 - Notification de sauvegarde - SavingWindow.scala

```

1 import ch.hevs.gdx2d.components.graphics.Polygon
2 import ch.hevs.gdx2d.lib.GdxGraphics
3 import com.badlogic.gdx.graphics.Color
4 import com.badlogic.gdx.math.Vector2
5
6 // Confirmation dialog shown when the "mop" (clear all) action is triggered
7
8 class AreYouSureWindow {
9     //Window Size
10    val firstWindowPoint: Vector2 = new Vector2(200, 960)
11    val windowHeight: Int = 60
12    val windowWidth: Int = 130
13
14    //Window Coordinates
15    val windowCo: Array[Vector2] = Array(
16        firstWindowPoint,
17        new Vector2(firstWindowPoint.x, firstWindowPoint.y + windowHeight),
18        new Vector2(firstWindowPoint.x + windowWidth, firstWindowPoint.y + windowHeight),
19        new Vector2(firstWindowPoint.x + windowWidth, firstWindowPoint.y)
20    )
21
22    //Button Margins
23    private val buttonMargin: Int = 10
24    private val buttonHeight: Int = 20
25    private val buttonWidth: Int = 50
26
27    //Yes Button Coordinates
28    private val firstButtonPoint: Vector2 = new Vector2(firstWindowPoint.x + buttonMargin, firstWindowPoint.y +
buttonMargin)
29    private val yesButton: Array[Vector2] = Array(
30        firstButtonPoint,
31        new Vector2(firstButtonPoint.x, firstButtonPoint.y + buttonHeight),
32        new Vector2(firstButtonPoint.x + buttonWidth, firstButtonPoint.y + buttonHeight),
33        new Vector2(firstButtonPoint.x + buttonWidth, firstButtonPoint.y)
34    )
35    //No Button Coordinates
36    private val noButtonPoint: Vector2 = new Vector2(firstButtonPoint.x + buttonWidth + buttonMargin, firstButtonPoint.y)
37    private val noButton: Array[Vector2] = Array(
38        noButtonPoint,
39        new Vector2(noButtonPoint.x, noButtonPoint.y + buttonHeight),
40        new Vector2(noButtonPoint.x + buttonWidth, noButtonPoint.y + buttonHeight),
41        new Vector2(noButtonPoint.x + buttonWidth, noButtonPoint.y)
42    )
43
44    private var answer: String = "" //Answer storage var
45
46    def drawWindow(g: GdxGraphics): Unit = {
47        //Saving old color setting
48        val oldColor = g.sbGetColor()
49
50        //Drawing the window
51        g.setColor(Color.BLACK)
52        g.drawFilledPolygon(new Polygon(windowCo), Color.WHITE)
53        g.drawPolygon(new Polygon(windowCo))
54        g.drawString(firstWindowPoint.x + 13, firstWindowPoint.y + windowHeight - buttonMargin, "Are you sure ?")
55        g.drawPolygon(new Polygon(yesButton))
56        g.drawString(firstButtonPoint.x + 12, firstButtonPoint.y + buttonHeight - 5, "Yes")
57        g.drawPolygon(new Polygon(noButton))
58        g.drawString(noButtonPoint.x + 15, noButtonPoint.y + buttonHeight - 5, "No")
59
60        //Switching back to old color
61        g.setColor(oldColor)
62    }
63
64    def onClick(x: Int, y: Int): Unit = {

```

```

65 //If the mouse is in the Yes button coordinates, answer = yes
66 if (x >= firstButtonPoint.x && x <= firstButtonPoint.x + buttonWidth &&
67     y >= firstButtonPoint.y && y <= firstButtonPoint.y + buttonHeight) {
68     answer = "yes"
69
70 //If the mouse is in the NO button coordinates, answer = no
71 } else if (x >= noButtonPoint.x && x <= noButtonPoint.x + buttonWidth &&
72     y >= noButtonPoint.y && y <= noButtonPoint.y + buttonHeight) {
73     answer = "no"
74 } else {
75     answer = ""
76 }
77 }
78
79 def getAnswer: String = {
80     answer
81 }
82 }

```

Listing 15 - Fenêtre de confirmation - AreYouSureWindow.scala

9.6 – Modes de jeu

```

1  import ch.hevs.gdx2d.components.bitmaps.BitmapImage
2  import ch.hevs.gdx2d.lib.GdxGraphics
3  import com.badlogic.gdx.graphics.Color
4
5  import scala.collection.mutable.ArrayBuffer
6
7  // Manages the mode menu: icon loading, mode switching
8  class MenuModesMachine {
9      private val modes: ArrayBuffer[MenuModes] = ArrayBuffer.empty
10     private val icons: ArrayBuffer[BitmapImage] = ArrayBuffer.empty
11     private var cm: String = "lines"
12     private var dm: String = "physic"
13     private var mm: String = "music"
14     loadModes()
15     private var firstRun: Boolean = true
16
17     // Load icon images for each menu mode (called once)
18     def loadIcons(): Unit = {
19         if (firstRun) {
20             for (mode <- modes) {
21                 val img = new BitmapImage(s"./icons/${mode.name}.png")
22                 icons.addOne(img)
23             }
24             firstRun = false
25         }
26     }
27
28     // Draw all mode buttons on the top-right of the screen
29     def drawModesMenu(g: GdxGraphics, width: Int, height: Int): Unit = {
30         val startPointH: Int = width - 20
31         val startPointW: Int = height + 30
32         val radius = 18
33
34         for (i <- modes.indices) {
35             if (modes(i).name == "music" || modes(i).name == "musicmute") {
36                 if (getMusicMode() == modes(i).name) {
37                     modes(i).x = startPointW
38                     modes(i).y = startPointH
39                     modes(i).radius = radius
40
41                     if (getMusicMode() == modes(i).name) {
42                         g.drawCircle(modes(i).x, modes(i).y, modes(i).radius, Color.BLUE)
43                     }
44                     g.drawTransformedPicture(startPointW, startPointH, 0, 0.05f, icons(i))
45                     startPointW += 50
46                 }
47             }
48         }
49     }
50 }

```

```

47     } else {
48         if (modes(i).name == "physic" || modes(i).name == "decoration") {
49             if (getDrawMode() == modes(i).name) {
50                 modes(i).x = startPointW
51                 modes(i).y = startPointH
52                 modes(i).radius = radius
53
54                 if (getDrawMode() == modes(i).name) {
55                     g.drawCircle(modes(i).x, modes(i).y, modes(i).radius, Color.BLUE)
56                 }
57                 g.drawTransformedPicture(startPointW, startPointH, 0, 0.05f, icons(i))
58                 startPointW += 50
59             }
60         } else {
61             modes(i).x = startPointW
62             modes(i).y = startPointH
63             modes(i).radius = radius
64
65             if (currentMode() == modes(i).name) {
66                 g.drawCircle(modes(i).x, modes(i).y, modes(i).radius, Color.BLUE)
67             }
68             if (getDrawMode() == modes(i).name) {
69                 g.drawCircle(modes(i).x, modes(i).y, modes(i).radius, Color.BLUE)
70             }
71             g.drawTransformedPicture(startPointW, startPointH, 0, 0.05f, icons(i))
72             startPointW += 50
73         }
74     }
75 }
76 }
77
78 def currentMode(): String = {
79     cm
80 }
81
82 def getDrawMode(): String = {
83     dm
84 }
85
86 def getMusicMode(): String = {
87     mm
88 }
89
90 // Check if the user clicked a menu button
91 // Returns true if a button was hit
92 def onMenuClick(x: Int, y: Int): Boolean = {
93     for (m <- modes) {
94         if (math.sqrt((x - m.x) * (x - m.x) + (y - m.y) * (y - m.y)) < m.radius) {
95             // Button hit -> switch mode
96             modeSwitcher(m.name)
97             return true
98         }
99     }
100     false
101 }
102
103 // Handle mode toggles (physic/decoration, music/mute) and direct mode selection
104 def modeSwitcher(m: String): Unit = {
105     if (m == "physic") {
106         if (dm == "physic") {
107             dm = "decoration"
108         } else {
109             dm = "physic"
110         }
111     } else if (m == "music") {
112         if (mm == "music") {
113             mm = "musicmute"
114         } else {
115             mm = "music"
116         }
117     }
118 }

```

```

117     } else {
118         cm = m
119     }
120 }
121
122 // All Menus / Toggles added
123 private def loadModes(): Unit = {
124     modes.addOne(MenuModes("free"))
125     modes.addOne(MenuModes("lines"))
126     modes.addOne(MenuModes("play"))
127     modes.addOne(MenuModes("eraser"))
128     modes.addOne(MenuModes("mop"))
129     modes.addOne(MenuModes("return"))
130     modes.addOne(MenuModes("save"))
131     modes.addOne(MenuModes("load"))
132     modes.addOne(MenuModes("physic"))
133     modes.addOne(MenuModes("decoration"))
134     modes.addOne(MenuModes("music"))
135     modes.addOne(MenuModes("musicmute"))
136 }
137 }

```

Listing 16 - Gestionnaire des modes du menu - MenuModesMachine.scala

```

1 // Single menu button (name, position, size)
2 case class MenuModes(name: String, var x: Int = 0, var y: Int = 0, var radius: Int = 10)

```

Listing 17 - Structure des modes du menu - MenuModes.scala

9.7 – Types

```

1 import scala.collection.mutable.ArrayBuffer
2
3 // Type alias: a free-draw line is a collection of lines segments
4
5 package object typesLibrary {
6     type Free = ArrayBuffer[Line] // A free draw is a combination of multiple lines
7 }

```

Listing 18 - Type alias pour les lignes - typesLibrary.scala

Références

- [1] libGDX, « libGDX - Java Game Development Framework ». Consulté le: 9 juin 2026. [En ligne]. Disponible sur: <https://libgdx.com/>
- [2] B. Cadez, « Line Rider ». Consulté le: 9 juin 2026. [En ligne]. Disponible sur: <https://www.linerider.com/>
- [3] Box2D, « Box2D - A 2D Physics Engine for Games ». Consulté le: 9 juin 2026. [En ligne]. Disponible sur: <https://box2d.org/>